

PCA --> college version

```
# import lib
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# load the data
data = pd.read_csv("car_condition.csv")
data.columns = ["buying", "maintenance", "doors", "persons", "lug_boot",
"safety", "class"]
print(data)

# check and handle null data
print(data.isnull().sum())

# features and target
features = data.drop(["class"], axis="columns")
target = data["class"]
print(features)

input()

# handle cat data
nfeatures = pd.get_dummies(features)
print(nfeatures)

input()

# features standardization
ss = StandardScaler()
sfeatures = ss.fit_transform(nfeatures)
print(sfeatures)
```

```
input()
```

```
pca = PCA(n_components=3)  
pfeatures = pca.fit_transform(sfeatures)  
print(pfeatures)
```

```
input()
```

```
# train and test  
x_train, x_test, y_train, y_test = train_test_split(pfeatures, target)
```

```
# model  
model = DecisionTreeClassifier()  
model.fit(x_train, y_train)
```

```
# performance  
print(data["class"].value_counts())  
y_pred = model.predict(x_test)  
cr = classification_report(y_test, y_pred)  
print(cr)
```

PCA --> corporate

```
# import lib
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline

# load the data
data = pd.read_csv("car_condition.csv")
data.columns = ["buying", "maintenance", "doors", "persons", "lug_boot",
"safety", "class"]
print(data)

# check and handle null data
print(data.isnull().sum())

# features and target
features = data.drop(["class"], axis="columns")
target = data["class"]
print(features)

# train and test
x_train, x_test, y_train, y_test = train_test_split(features, target)

# model
model = make_pipeline(
    OneHotEncoder(sparse_output=False),
    StandardScaler(with_mean=False),
    PCA(n_components=4),
    DecisionTreeClassifier()
)
model.fit(x_train, y_train)
```

```
# performance
print(data["class"].value_counts())
y_pred = model.predict(x_test)
cr = classification_report(y_test, y_pred)
print(cr)
```

ML with Kamal Sir - 8369084928